Reproducing GTTA on BFCLv3: What I Saw

Natnael Bekele

natnaelbekele142@gmail.com · nathybekele.github.io · nathyBekele · LinkedIn · SA Inspector (live)

SUMMARY

1. **Dynamics Grounding (DG) works as reported:** +3.5 to +4.0 points in all three of my runs, vs. +3.5 in the paper.
2. **Syntactic Alignment (SA) didn't change how many tasks were solved**, at the paper's default or best setting.
3. **A possible reason:** in the released code, SA's step gets smaller as the prompt gets longer.

1 What I did

Test-Time Adaptation for LLM Agents via Environment Interaction (GTTA; Chen et al., ICLR 2026) proposes two ways for an agent to adapt to a new environment. **Syntactic Alignment (SA)** learns a small vector δ added to the model's last hidden layer, with a few Adam steps on the current prompt. **Dynamics Grounding (DG)** lets the agent explore, writes what it saw down as rules, and adds them to the prompt.

I reran both with Qwen2.5-14B-Instruct on the authors' 200 BFCLv3 multi-turn tasks (commit `24ba304`), scored with BFCL's own checker. I reimplemented SA from the released `vllm/model_runner.py` with the same schedule; for DG, Qwen-14B explored and wrote its own rules (the paper used GPT-4.1/GPT-4o/o3). All experiments cost about $13 on rented GPUs.

One observation guided the rest: the logits are $(H+\delta)W^T = HW^T + \delta W^T$, so SA adds a fixed score to every word in the vocabulary, and I can read off exactly which words it pushes up.

2 Findings

DG works as reported. Without adaptation the agent solves 17.5–18.5% of tasks; with DG rules, 21.0–22.0% (Figure 1). Two of my runs hit the paper's numbers exactly, and the gain is bigger than the 1-point noise between identical runs.

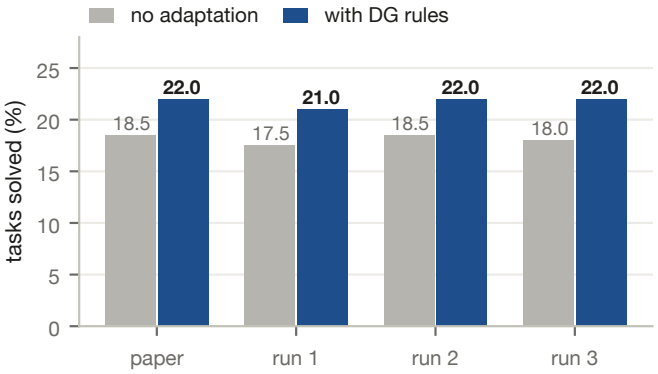


Figure 1. Tasks solved with and without DG rules, on all 200 tasks. My three runs use identical prompts.

SA didn't change how many tasks were solved. At the paper's default setting it changed generated words in 53 of 200 tasks but never an outcome (+0.0 points). At the paper's best setting it gained +0.5 points (5 wins, 4 losses; 95% CI −2.5 to +3.5), within the noise.

A possible reason: the step shrinks as the prompt grows. The released code averages the gradient over the prompt twice (divide by the number of tokens, then `.mean(0)`), which equals the exact gradient divided by the prompt length. On BFCL's 4K–8K-token prompts every coordinate falls below Adam's $\varepsilon = 10^{-6}$, so the step is about 64× smaller than Adam would normally take, and it keeps shrinking with length (Figure 2).

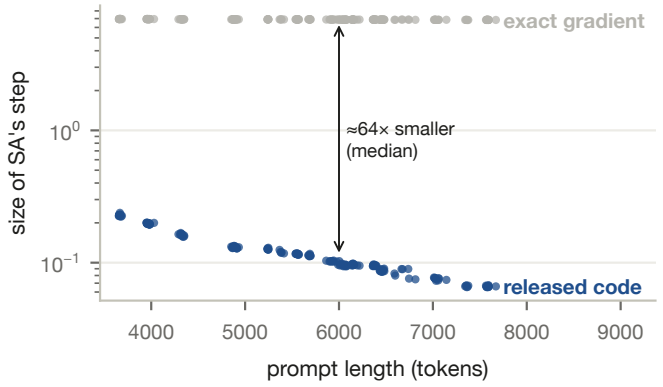


Figure 2. Size of SA's first step (‖δ‖, lr 0.1) on each of the 200 prompts, with the released code and with the exact gradient.

What SA pushes up is the prompt's most frequent text: punctuation, newlines, and “Note”, because BFCL appends “Note that the provided function is in Python 3 syntax” to every function (Figure 3). Only about 8% of these words appear in the correct answer, and SA doesn't make the correct first call more likely at either setting.

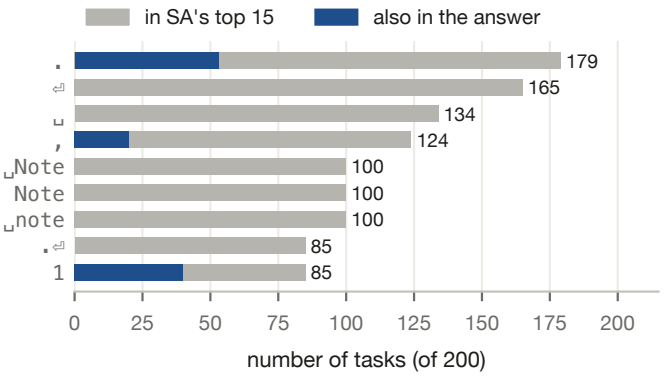


Figure 3. Words most often among SA's 15 most-boosted, across the 200 tasks, for a full-size step. The released code's tiny step leans the same way.

SA together with DG. DG's rules make prompts 36% longer, which shrinks SA's step further (median ‖δ‖ 0.108 → 0.070). This may be one reason the paper's SA+DG combination (21.0%) didn't beat DG alone (22.0%).

3 How I got there

- **DG failed silently at first.** Qwen often writes several calls per step, BFCL rejected them, and 55 of 80 exploration logs came out empty. Running the first valid call (during exploration only) fixed it.
- **Noise first.** Identical runs moved by about 1 point, so I repeated DG three times before trusting the gain.
- **A shortcut for SA.** A task where SA changes no generated word must end the same way, so I only reran the tasks it changed.

4 Limitations

- SA ran in my own PyTorch loop, not the original vLLM server, so I compare it only with no SA in the same loop.
- One benchmark; WebArena's prompts may behave differently.
- DG used Qwen instead of GPT-4.1/GPT-4o/o3 to write rules.

5 Open questions

- Which user-turn masking setting (`MASK_TOKENS`) was used for the reported runs?
- Was the larger learning rate (1.0) meant to make up for the small step?

6 Check it yourself

The SA Inspector runs SA live on a BFCL task or your own text, and the DG Inspector shows every DG rule and trajectory. Code, per-task results and a log of every decision: github.com/nathyBekele/reproducing-gtta.